

École Polytechnique de Thiès (EPT)  
Cloud & DevOps Bootcamp

# Déploiement d'Ollama et Open WebUI

avec Docker Compose et Kubernetes (Minikube)

Rapport technique

**Étudiant :** Abdoulaye Sy Ndaw  
**Formateur :** Ibrahima MBENGUE  
**Formation :** Cloud & DevOps Bootcamp  
**Année :** 2026

## 1 Rôle de chaque ressource Kubernetes

Le projet déploie deux applications (Ollama et Open WebUI) dans le namespace `llm-app`, à partir des manifests situés dans `k8s/base/`. Chaque ressource a un rôle précis dans le cycle de vie et l'exposition de ces applications.

### 1.1 Namespace

Le Namespace `llm-app` isole toutes les ressources du projet du reste du cluster Minikube. Toutes les ressources suivantes (ConfigMap, PVC, Deployment, Service, NetworkPolicy, PodDisruptionBudget) sont créées dans ce namespace, ce qui permet de gérer, lister et supprimer le projet entier sans toucher aux autres workloads du cluster.

```
apiVersion: v1
kind: Namespace
metadata:
  name: llm-app
```

### 1.2 ConfigMap

Deux ConfigMap stockent les variables de configuration en dehors des images Docker. `ollama-config` définit `OLLAMA_HOST=0.0.0.0` pour qu'Ollama écoute sur toutes les interfaces réseau du pod. `webui-config` définit `OLLAMA_BASE_URL=http://ollama-service:11434`, l'URL que Open WebUI utilise pour contacter Ollama. Les deux Deployments injectent ces valeurs via `envFrom.configMapRef`.

### 1.3 PersistentVolumeClaim

Deux PVC fournissent le stockage persistant : `ollama-pvc` (20Gi, monté sur `/root/.ollama`) pour les modèles LLM, et `webui-pvc` (2Gi, monté sur `/app/backend/data`) pour les comptes utilisateurs et l'historique de conversation d'Open WebUI. Le détail de leur nécessité est traité en section 3.

### 1.4 Deployment

Les Deployments `ollama` et `open-webui` gèrent le cycle de vie des pods : création, remplacement en cas de crash, rollout lors d'une mise à jour d'image. Chaque Deployment tourne avec `replicas: 1` dans ce projet, et définit les probes, les volumeMounts et les limites de ressources CPU/mémoire du conteneur.

### 1.5 Service ClusterIP (ollama-service)

`ollama-service` est un Service de type ClusterIP qui expose le port 11434 des pods Ollama uniquement à l'intérieur du cluster. Il ne fournit pas d'IP externe accessible depuis l'extérieur de Minikube, ce qui correspond à son rôle : point d'entrée interne pour Open WebUI.

```
apiVersion: v1
kind: Service
metadata:
  name: ollama-service
spec:
  selector:
    app: ollama
  ports:
    - port: 11434
      targetPort: 11434
  type: ClusterIP
```

### 1.6 Service NodePort (webui-service)

`webui-service` expose le port 8080 du pod Open WebUI sur le port 30080 de chaque nœud du cluster, via `type: NodePort`. C'est ce qui permet d'accéder à l'interface web depuis l'extérieur de Minikube, par exemple avec `minikube service webui-service -n llm-app` ou directement sur `<node-ip>:30080`.

### 1.7 NetworkPolicy

`ollama-network-policy` restreint le trafic entrant vers les pods labellisés `app: ollama`. Seuls les pods labellisés `app: open-webui` peuvent atteindre le port 11434. Tout autre pod du cluster, même dans le même namespace, ne peut pas contacter Ollama directement. C'est une mesure de sécurité réseau au niveau du cluster, indépendante de toute authentification applicative.

### 1.8 PodDisruptionBudget

Deux PDB (`ollama-pdb` et `webui-pdb`) fixent `minAvailable: 1` pour chaque application. Lors d'une opération de maintenance planifiée sur le cluster (drain d'un nœud, mise à jour Kubernetes), l'API Kubernetes empêche l'éviction d'un pod si cela fait passer le nombre de pods disponibles sous ce seuil. Avec un seul replica par Deployment dans ce projet, le PDB garantit surtout que le scheduler ne vide pas le pod sans réflexion pendant une opération de cluster.

## 2 Communication entre les services

Le flux de communication suit un chemin fixe, défini par le DNS interne de Kubernetes et la NetworkPolicy.

1. Open WebUI lit la variable `OLLAMA_BASE_URL` (injectée depuis `webui-config`) et envoie ses requêtes API vers `http://ollama-service:11434`.
2. Le CoreDNS du cluster résout le nom `ollama-service` (format court, car les deux pods sont dans le même namespace `llm-app`) vers l'IP virtuelle (ClusterIP) attribuée au Service `ollama-service`.
3. Le kube-proxy route ce trafic vers un des pods backend du Service, sélectionnés via le label `app: ollama`, sur le port cible 11434.
4. La NetworkPolicy `ollama-network-policy` vérifie que la requête provient bien d'un pod avec le label `app: open-webui`. C'est le seul flux entrant autorisé vers Ollama sur ce port ; tout autre trafic est bloqué au niveau du plugin réseau CNI.
5. Côté utilisateur, l'accès se fait sur le NodePort 30080 exposé par `webui-service`. C'est le seul point d'entrée externe du système : Ollama n'est jamais exposé en dehors du cluster.

Ce découpage sépare clairement le trafic interne (Open WebUI vers Ollama, contrôlé par la NetworkPolicy) du trafic externe (utilisateur vers Open WebUI, via NodePort).

### 3 Pourquoi les PersistentVolumeClaims sont nécessaires

Un pod Kubernetes est éphémère par conception. Quand un pod est recréé (crash, rollout, éviction par le scheduler), son système de fichiers local repart de zéro à partir de l'image du conteneur. Sans volume persistant, tout ce qui a été écrit dans le conteneur disparaît.

**Ollama** télécharge et stocke ses modèles LLM dans `/root/.ollama`. Sur ce projet, trois modèles sont utilisés : `llama3`, `mistral` et `deepseek`. Ces modèles pèsent plusieurs gigaoctets chacun, ce qui justifie les 20Gi alloués au `ollama-pvc`. Sans ce PVC, chaque redémarrage du pod Ollama forcerait un nouveau téléchargement complet des trois modèles avant de pouvoir répondre à la moindre requête, ce qui rend le service indisponible pendant plusieurs minutes à chaque redéploiement.

**Open WebUI** stocke les comptes utilisateurs, les paramètres et l'historique des conversations dans `/app/backend/data`, monté sur `webui-pvc` (2Gi). Sans ce PVC, un simple rollout du Deployment `open-webui` effacerait tous les comptes et tout l'historique de chat.

Avec les PVC en place, le stockage est découplé du cycle de vie du pod. Kubernetes réattache le même volume au nouveau pod après un redémarrage ou une mise à jour d'image, et les données (modèles téléchargés, comptes, historique) survivent à l'opération.

### 4 Rôle des probes

Les trois Deployments définissent chacun trois types de probes, avec des mécanismes différents selon la nature de l'application.

#### 4.1 startupProbe

Vérifie qu'Ollama a fini de démarrer avant que les autres probes ne prennent le relais. Configurée avec `failureThreshold: 30` et `periodSeconds: 10`, ce qui laisse jusqu'à 5 minutes (30 tentatives × 10 secondes) au conteneur pour devenir opérationnel avant que Kubernetes ne le considère en échec. Ce délai est nécessaire car le chargement initial d'Ollama et de ses dépendances peut prendre du temps, surtout sur les ressources limitées de Minikube.

```
startupProbe:
  exec:
    command: ["ollama", "list"]
  failureThreshold: 30
  periodSeconds: 10
  timeoutSeconds: 10
```

#### 4.2 readinessProbe

Détermine si le pod est prêt à recevoir du trafic via le Service. Tant que `ollama list` échoue, le pod est retiré des endpoints de `ollama-service`, et Open WebUI ne peut pas lui envoyer de requêtes. Cela évite d'envoyer des prompts vers une instance Ollama pas encore prête à les traiter.

#### 4.3 livenessProbe

Détecte un pod bloqué (deadlock, process figé) qui répond toujours aux vérifications TCP de base mais n'exécute plus rien correctement. Si `ollama list` échoue de façon répétée après le démarrage, kubelet tue et redémarre le conteneur automatiquement, sans intervention manuelle.

#### 4.4 timeoutSeconds

Fixé à 10 secondes sur toutes les probes d'Ollama, au lieu de la valeur par défaut de 1 seconde. La commande `ollama list` interroge le moteur d'inférence et peut prendre plus d'une seconde à répondre sur les ressources CPU limitées d'un environnement Minikube local, surtout quand un modèle est en cours de chargement en mémoire. Avec le timeout par défaut, les probes auraient échoué en continu par faux positifs.

#### 4.5 Cas d'Open WebUI

Les probes d'`open-webui` utilisent `httpGet` sur le chemin `/health` et le port 8080, au lieu d'un `exec`. Contrairement à Ollama (CLI qui expose une commande `ollama list`), Open WebUI est une application web qui expose nativement un endpoint HTTP de santé. Un `httpGet` est plus léger qu'un `exec` (pas de fork de process dans le conteneur à chaque probe) et correspond directement au protocole que l'application sert déjà.

### 5 Différence entre Docker Compose et Kubernetes

#### 5.1 Docker Compose

Docker Compose est un outil local orienté développement. Le fichier `docker-compose.yaml` de ce projet définit les deux services (`ollama`, `open-webui`), leurs volumes, leur réseau bridge et leurs `healthcheck` en une trentaine de lignes. Le démarrage se fait avec une seule commande :

```
docker compose up -d
```

Il n'y a ni haute disponibilité, ni self-healing au niveau orchestrateur : le `healthcheck` du conteneur `ollama` permet à `open-webui` d'attendre qu'Ollama soit `service_healthy` avant de démarrer (via `depends_on`), mais si le conteneur `ollama` crashe de façon répétée, Docker Compose ne fait que le relancer selon la police `restart: unless-stopped`, sans logique de scheduling, de scaling ou de répartition sur plusieurs hôtes.

#### 5.2 Kubernetes

Kubernetes ajoute une couche d'orchestration complète : self-healing via les probes et les Deployment controllers, scaling horizontal (non utilisé ici avec `replicas: 1`, mais disponible nativement), gestion fine des ressources CPU/mémoire par conteneur (`requests/limits`), health checks à trois niveaux (`startup`, `readiness`, `liveness`) contre un seul `healthcheck` par service dans Compose, stockage persistant natif découplé du pod via PVC, et NetworkPolicy pour isoler le trafic entre applications.

#### 5.3 Choix sur ce projet

Docker Compose sert pour le développement local rapide, sans la complexité d'un cluster. Kubernetes (Minikube) sert à simuler un environnement de production : namespace dédié, probes ajustées aux contraintes réelles de l'infrastructure, NetworkPolicy pour le contrôle d'accès, PDB pour la résilience lors des maintenances. Les deux fichiers (`docker-compose.yaml` et `k8s/base/`) coexistent dans le dépôt et couvrent chacun une étape du cycle de vie du projet.